
KshetraAI

Judge Reference Appendix

Explainable Field-Force Intelligence
for Agricultural Sales Operations

Stage 1 Submission

Syngenta Hackathon

May 2026

Contents

1	Document Scope	1
1.1	Reference Roadmap	1
2	Data Foundation	2
2.1	Purpose	2
2.2	Implementation Summary	2
2.3	How It Works	2
2.4	Data Sources	2
2.5	Demo Evidence	3
2.6	Current Public Signal Truth	3
2.7	Verification	3
2.8	Current Limits	3
2.9	Judge Takeaway	4
3	Feature Generation	5
3.1	Purpose	5
3.2	Implementation Summary	5
3.3	How It Works	5
3.4	Generated Feature Views	5
3.5	Signal Categories	5
3.6	Demo Evidence	6
3.7	Verification	6
3.8	Current Limits	6
3.9	Judge Takeaway	6
4	Dynamic Prioritization	7
4.1	Purpose	7
4.2	Implementation Summary	7
4.3	How It Works	7
4.4	Scoring Logic	7
4.5	Demo Evidence	8
4.6	Verification	8

4.7	Current Limits	8
4.8	Judge Takeaway	9
5	Contextual Decision Engine	10
5.1	Purpose	10
5.2	Implementation Summary	10
5.3	How It Works	10
5.4	Rule Areas	10
5.5	Demo Evidence	11
5.6	Verification	11
5.7	Current Limits	11
5.8	Judge Takeaway	12
6	Anomaly and Opportunity Detection	13
6.1	Purpose	13
6.2	Implementation Summary	13
6.3	How It Works	13
6.4	Detector Areas	13
6.5	Demo Evidence	14
6.6	Verification	14
6.7	Current Limits	14
6.8	Judge Takeaway	15
7	Explainability and Trust	16
7.1	Purpose	16
7.2	Implementation Summary	16
7.3	How It Works	16
7.4	Demo Evidence	16
7.5	Verification	17
7.6	Current Limits	17
7.7	Judge Takeaway	17
8	Outcome Learning and Feedback	18
8.1	Purpose	18
8.2	Implementation Summary	18
8.3	How It Works	18
8.4	Outcome Fields	18
8.5	Demo Evidence	19

8.6	Verification	19
8.7	Current Limits	19
8.8	Judge Takeaway	19
9	FastAPI Backend Integration	20
9.1	Purpose	20
9.2	Implementation Summary	20
9.3	How It Works	20
9.4	API Contract	20
9.5	Demo Evidence	21
9.6	Swagger / OpenAPI	21
9.7	Verification	21
9.8	Current Limits	22
9.9	Judge Takeaway	22
10	Frontend Dashboard and Workflow Layer	23
10.1	Purpose	23
10.2	Implementation Summary	23
10.3	How It Works	23
10.4	Frontend Workflow	23
10.5	Demo Evidence	24
10.6	Verification	24
10.7	Current Limits	24
10.8	Judge Takeaway	25
11	Demo Integration, Testing, and Final Polish	26
11.1	Purpose	26
11.2	Implementation Summary	26
11.3	How It Works	26
11.4	Fixed Demo Scenario	26
11.5	Demo Evidence	27
11.6	Verification	27
11.7	Current Limits	28
11.8	Judge Takeaway	28

1 Document Scope

This appendix consolidates the judge-reference notes for KshetraAI into one reviewable document. It is intended to support the presentation deck with implementation-grounded detail.

The claims in this appendix are limited to the current codebase, generated artifacts, sample outputs, and verified demo workflow. Where a capability is a foundation or future scope, it is labeled that way.

1.1 Reference Roadmap

Section	What It Covers
Data Foundation	source boundaries, schemas, public-data readiness
Feature Generation	normalized decision signals and generated feature views
Dynamic Prioritization	visit ranking, component weights, and demo scenario output
Contextual Decision	next best action rules and recommendation evidence
Anomaly Detection	alerts, severity, confidence, and current calibration limits
Explainability	evidence-backed reasoning for priority, recommendations, and alerts
Outcome Learning	outcome capture and human-governed feedback foundation
API Integration	FastAPI contracts and processed-output serving boundary
Frontend Workflow	React dashboard path and UI behavior
Demo Integration	deterministic scenario, sample outputs, and acceptance checks

2 Data Foundation

2.1 Purpose

Build a reliable data foundation for KshetraAI by separating raw source data from processed, demo-safe operational views.

2.2 Implementation Summary

- Implemented schema definitions for the company-provided internal dataset.
- Created a deterministic Build 01 pipeline for loading, validating, normalizing, and joining source files.
- Established a private raw data boundary through ignored `private-data/`.
- Established a public raw data boundary through ignored `public-data/`.
- Added a public-data processing workflow for weather, crop context, NDVI metadata, and pest references.

2.3 How It Works

Internal CSV files are validated against schema definitions, normalized into canonical forms, joined into feature-ready views, and written as processed outputs. Public data is processed separately into public signal tables so it can be integrated later without mixing raw private and public sources.

2.4 Data Sources

Internal schemas implemented in code:

- `reps_territory.csv`
- `retailers.csv`
- `retailer_visit_log.csv`
- `retailer_inventory_weekly.csv`
- `retailer_pos.csv`
- `growers.csv`
- `digital_funnel_weekly.csv`
- `whatsapp_campaign.csv`

Public-data foundation:

- weather signals from fetched Open-Meteo data
- controlled crop-stage context
- Sentinel-2 scene metadata / NDVI reference tables
- pest surveillance source references

2.5 Demo Evidence

Canonical processed views include:

- `representatives`
- `territories`
- `retailers`
- `growers`
- `visit_entities`
- `crop_context`
- `retailer_pos_clean`
- `retailer_inventory_weekly_clean`
- `retailer_visit_log_clean`
- `campaign_engagement_clean`

Public processed outputs include:

- `datasets/processed/public/weather_signals.csv`
- `datasets/processed/public/crop_context.csv`
- `datasets/processed/public/ndvi_scene_inventory.csv`
- `datasets/processed/public/ndvi_signals.csv`
- `datasets/processed/public/pest_source_references.csv`
- `datasets/processed/public/pest_signals.csv`

2.6 Current Public Signal Truth

- Weather data is usable as public signal data.
- Crop-stage context is curated/controlled for the demo.
- NDVI is currently metadata/reference level, not raster-derived crop health scoring.
- Pest data is currently reference-level, not active live pest outbreak detection.
- Public processed tables exist, but are not fully merged into the main private/public feature generation run yet.

2.7 Verification

Relevant implementation areas:

- `backend/data/schemas/dataset_schemas.py`
- `backend/pipelines/pipeline_runner.py`
- `scripts/process_public_data.py`
- `tests/test_public_data_processing.py`

2.8 Current Limits

- Raw private data is local only and should not be shared.
- Live public API calls are not used during the judge demo.
- Public NDVI and pest signals are foundations, not full production-grade integrations.

2.9 Judge Takeaway

KshetraAI starts from a controlled, schema-driven data foundation with clear privacy boundaries and a dedicated path for public-domain signal integration.

3 Feature Generation

3.1 Purpose

Convert canonical operational data into normalized, interpretable feature signals that downstream engines can score, rank, explain, and expose through the API.

3.2 Implementation Summary

- Implemented a feature registry with 18 registered features.
- Built feature generators across agronomic, sales, inventory, relationship, competitive, and travel categories.
- Produced multiple feature views for different downstream modules.
- Preserved feature metadata so each signal remains interpretable.

3.3 How It Works

The feature pipeline reads canonical views, computes normalized feature scores, and writes module-specific feature views. These feature views become the shared input layer for prioritization, recommendations, anomaly detection, and explanations.

3.4 Generated Feature Views

Current processed feature artifacts:

- `combined_feature_view.csv`: 10,000 rows, 22 columns
- `priority_feature_view.csv`: 10,000 rows, 22 columns
- `contextual_feature_view.csv`: 10,000 rows, 13 columns
- `anomaly_feature_view.csv`: 10,000 rows, 9 columns
- `feature_registry.csv`: 18 rows, 10 columns

3.5 Signal Categories

Implemented feature categories:

- agronomic signals
- sales opportunity signals
- inventory need signals
- relationship / engagement signals
- competitive pressure signals
- travel cost signals

3.6 Demo Evidence

Feature generation supports the later demo outputs:

- the priority engine reads feature scores to rank entities
- contextual rules use feature values to trigger recommendations
- anomaly detection reads feature values against baselines
- explanations map final outputs back to evidence signals

3.7 Verification

Relevant implementation areas:

- `backend/features/feature_registry.py`
- `backend/features/feature_pipeline.py`
- `backend/features/agronomic_features.py`
- `backend/features/sales_features.py`
- `backend/features/inventory_features.py`
- `backend/features/relationship_features.py`
- `backend/features/competitor_features.py`
- `backend/features/travel_features.py`
- `tests/test_build02_feature_registry.py`
- `tests/test_build02_feature_pipeline.py`

Focused Build 02 tests were verified during development.

3.8 Current Limits

- Public processed signal tables are not yet fully merged into the main final feature generation run.
- Competitive and travel signals depend on matching input availability.
- Feature values are deterministic engineering signals, not trained ML predictions.

3.9 Judge Takeaway

KshetraAI turns raw operational records into a reusable, explainable feature layer that powers the rest of the intelligence workflow.

4 Dynamic Prioritization

4.1 Purpose

Generate a ranked daily visit plan so a field representative can focus on the most relevant visit first.

4.2 Implementation Summary

- Implemented component scorers for major operational signal groups.
- Implemented weighted priority scoring.
- Implemented priority classification thresholds.
- Implemented stable ranking logic.
- Generated a processed `ranked_visit_list.csv`.
- Exposed ranked visits through the `/daily-plan` API and frontend daily-plan view.

4.3 How It Works

The priority engine reads feature views, calculates component scores, applies configured weights, classifies the final score into a priority level, and ranks entities deterministically.

4.4 Scoring Logic

Current component weights:

- agronomic: 0.30
- sales: 0.25
- inventory: 0.20
- relationship: 0.10
- competitive: 0.10
- travel cost: -0.05

Priority thresholds:

- Critical: 80+
- High: 65+
- Medium: 50+
- Low: 0+

4.5 Demo Evidence

Current ranked output:

- `datasets/processed/ranked_visit_list.csv`
- 10,000 rows
- 25 columns

Fixed demo scenario:

`rep_id: REP_0164`
`territory_id: TER_0164`
`date: 2026-05-17`

Current daily-plan sample:

- total rows for scenario: 23
- page size: 3
- top entity: `RTL_01300`
- priority score: 37.1125
- priority level: Low
- main reason: `Highest signal: inventory need (92.5)`

4.6 Verification

Relevant implementation areas:

- `backend/engines/component_scorers.py`
- `backend/engines/scoring_engine.py`
- `backend/engines/priority_classifier.py`
- `backend/engines/ranking_engine.py`
- `backend/engines/priority_engine.py`
- `tests/test_build03_priority_config.py`
- `tests/test_build03_priority_scoring.py`
- `tests/test_build03_priority_classifier.py`
- `tests/test_build03_priority_engine_integration.py`

Focused Build 03 tests were verified during development.

4.7 Current Limits

- The current generated demo data produces Low absolute priority levels for all rows.
- The engine supports Critical, High, Medium, and Low levels, but the current demo feature values do not reach the higher thresholds.
- The daily-plan API reads generated ranked outputs rather than recomputing all priority logic live per request.

4.8 Judge Takeaway

KshetraAI has a deterministic priority engine that ranks visits by traceable component scores, even when the current demo dataset produces low absolute scores.

5 Contextual Decision Engine

5.1 Purpose

Generate a next best action for a selected visit entity so the field representative knows what to discuss or inspect during the visit.

5.2 Implementation Summary

- Implemented deterministic rule matching for contextual recommendations.
- Added controlled YAML rule definitions.
- Generated recommendation outputs and advisory outputs.
- Preserved rule-match and recommendation trace logs.
- Exposed selected recommendations through the `/recommendations/{entity_id}` API.

5.3 How It Works

The contextual engine reads contextual feature rows, evaluates controlled rules, creates recommendation records for matching rules, and writes advisory/recommendation outputs with confidence and evidence fields.

5.4 Rule Areas

Implemented rule categories include:

- agronomic risk
- inventory replenishment
- sales opportunity
- relationship / campaign engagement
- competitive pressure

Current rule set:

- 10 YAML rules
- deterministic rule evaluation
- no black-box recommendation model

5.5 Demo Evidence

Current processed recommendation output:

- `datasets/processed/recommendation_outputs.csv`
- 10,041 rows
- 14 columns

Current rule distribution:

- `NO_CONTEXTUAL_RULE_MATCH`: 9,300 rows
- `SALES_PURCHASE_HISTORY`: 391 rows
- `RELATIONSHIP_CAMPAIGN`: 226 rows
- `INVENTORY_REPLENISHMENT`: 102 rows
- `INVENTORY_FAST_MOVING`: 20 rows
- `SALES_SEASONAL`: 2 rows

Sample recommendation for `RTL_01300`:

`risk_or_opportunity`: Possible fast-moving stock pressure
`recommended_product_category`: Relevant Seasonal SKU
`confidence_level`: Medium

Recommended actions:

- `review_fast_moving_sku_availability`
- `check_reorder_timing`
- `plan_inventory_follow_up`

5.6 Verification

Relevant implementation areas:

- `backend/engines/contextual_decision_engine.py`
- `backend/engines/rule_matcher.py`
- `backend/engines/recommendation_engine.py`
- `backend/engines/advisory_engine.py`
- `backend/engines/action_selector.py`
- `backend/config/contextual_rules.yaml`
- `tests/test_build04_recommendation_engine.py`
- `tests/test_build04_advisory_action_selection.py`

Focused Build 04 tests were verified during development.

5.7 Current Limits

- Most current generated rows do not trigger contextual rules.
- The current API returns one recommendation response for a selected entity, not the full advisory bundle.
- Recommendations are rule-based and deterministic, not ML-generated.

5.8 Judge Takeaway

KshetraAI converts feature signals into concrete next best actions while keeping the recommendation logic rule-based and auditable.

6 Anomaly and Opportunity Detection

6.1 Purpose

Detect operational exceptions such as stock-out risk, sales decline, and demand spikes so the representative can respond before the opportunity or risk is missed.

6.2 Implementation Summary

- Implemented deterministic anomaly and opportunity detectors.
- Added configured baseline comparisons.
- Generated alert outputs and anomaly trace logs.
- Classified alerts by severity and confidence.
- Exposed alerts through the `/alerts` API and frontend alert panel.

6.3 How It Works

The anomaly engine reads feature rows, compares selected signals against configured baselines and thresholds, emits alert records, and preserves supporting evidence in a trace log.

6.4 Detector Areas

Implemented detector categories:

- agronomic
- sales opportunity / sales risk
- inventory
- competitive
- operational

Current visible alert types:

- stock-out risk
- sales decline warning
- demand spike opportunity

6.5 Demo Evidence

Current processed outputs:

- datasets/processed/anomaly_baseline_view.csv: 10,000 rows, 32 columns
- datasets/processed/anomaly_alerts.csv: 8,827 rows, 13 columns
- datasets/processed/anomaly_trace_log.csv: 8,827 rows, 16 columns

Alert type distribution:

- sales decline warning: 7,006 rows
- possible stock-out risk: 1,634 rows
- demand spike opportunity: 187 rows

Fixed demo territory:

territory_id: TER_0164
alert_count: 19

Top sample alert:

entity_id: RTL_01300
alert_type: Possible stock-out risk
severity_score: 78.5
severity_level: High
confidence_level: High

6.6 Verification

Relevant implementation areas:

- backend/anomaly/anomaly_engine.py
- backend/anomaly/baseline_builder.py
- backend/anomaly/detectors.py
- backend/anomaly/alert_generator.py
- backend/api/services/anomaly_service.py
- tests/test_build05_anomaly_engine_integration.py
- tests/test_build05_alert_generation.py

Focused Build 05 tests were verified during development.

6.7 Current Limits

- Alert volume is high because baselines and thresholds are prototype-level.
- Alert calibration should not be presented as production-tuned.
- The API reads generated alert outputs rather than detecting anomalies live per request.

6.8 Judge Takeaway

KshetraAI has a working deterministic alerting layer that surfaces operational risk and opportunity signals with severity, confidence, and evidence.

7 Explainability and Trust

7.1 Purpose

Make priority, recommendation, and anomaly outputs understandable by mapping them back to evidence, confidence, and human-readable reasoning.

7.2 Implementation Summary

- Implemented evidence mapping for priority, recommendation, and anomaly outputs.
- Generated explanation outputs and trace logs.
- Added confidence reasoning and safety validation.
- Built frontend explanation presentation that groups evidence for readability.
- Exposed explanations through `/explanations/{entity_id}`.

7.3 How It Works

The explainability layer takes generated intelligence outputs, extracts supporting evidence fields, applies deterministic explanation templates, assigns confidence information, and writes explanation records and traces.

7.4 Demo Evidence

Current explanation outputs are generated for the selected demo entity:

`entity_id: RTL_01300`

Current processed outputs:

- `datasets/processed/evidence_view.csv`: 6 rows, 9 columns
- `datasets/processed/explanation_outputs.csv`: 6 rows, 11 columns
- `datasets/processed/explanation_trace_log.csv`: 6 rows, 8 columns

Explanation type distribution:

- recommendation: 3
- anomaly: 2
- priority: 1

Confidence distribution:

- High: 3
- Medium: 3

Sample explanation types available:

- why the entity was ranked
- why recommendations were suggested
- why alerts were raised

7.5 Verification

Relevant implementation areas:

- `backend/explainability/evidence_mapper.py`
- `backend/explainability/explanation_engine.py`
- `backend/explainability/confidence_explainer.py`
- `backend/api/services/explainability_service.py`
- `frontend/components/ExplanationPanel.tsx`
- `tests/test_build06_evidence_mapper.py`
- `tests/test_build06_explainability_integration.py`
- `tests/test_build06_explanation_generation.py`

Focused Build 06 tests were verified during development.

7.6 Current Limits

- The current generated explanation set is focused on the selected demo entity, not every entity in the full dataset.
- Some explanation text is prototype wording, such as “current severity”.
- Explanations are deterministic template-based outputs, not natural-language generation from an LLM.

7.7 Judge Takeaway

KshetraAI does not hide recommendations behind opaque scoring; the demo entity has visible evidence for priority, recommendation, and alert decisions.

8 Outcome Learning and Feedback

8.1 Purpose

Capture field outcomes so recommendations and alerts can later be measured, reviewed, and improved under human governance.

8.2 Implementation Summary

- Implemented outcome submission normalization.
- Implemented outcome logging logic.
- Implemented feedback signal processing.
- Implemented performance metric and analytics modules.
- Implemented recalibration signal generation with human-review constraints.
- Exposed outcome submission through `POST /outcomes`.

8.3 How It Works

The outcome layer accepts a structured visit result, validates required fields, normalizes the outcome, and produces feedback signals that can support future analytics or human-reviewed recalibration.

8.4 Outcome Fields

The current outcome flow captures:

- recommendation ID
- entity ID
- rep ID
- visit completed
- recommendation followed
- sale made
- order placed
- order value
- alert validated
- feedback category
- rep feedback
- optional alert ID

8.5 Demo Evidence

Sample outcome response:

```
status: success
message: Outcome recorded successfully.
outcome_id: OUTCOME_RTL_01300_POSSIBLE_FAST_MOVING_STOCK_PRESSURE
```

Frontend outcome capture is available in:

```
frontend/pages/OutcomeSubmission.tsx
frontend/components/OutcomeForm.tsx
```

Backend outcome service is available in:

```
backend/api/services/outcome_service.py
```

8.6 Verification

Relevant implementation areas:

- backend/learning/outcome_logger.py
- backend/learning/feedback_processor.py
- backend/learning/metrics_tracker.py
- backend/learning/performance_analytics.py
- backend/learning/recalibration_engine.py
- backend/api/routes/outcome_routes.py
- backend/api/schemas/outcome_schema.py
- tests/test_build07_outcome_logger.py
- tests/test_build07_feedback_processor.py
- tests/test_build07_metrics_analytics.py
- tests/test_build07_recapibration_engine.py
- tests/test_build08_outcome_routes.py

Focused Build 07 and outcome API tests were verified during development.

8.7 Current Limits

- The current repository does not contain a persistent historical outcome dataset.
- Generated performance metric artifacts are not part of the current demo outputs.
- Recalibration signals are human-review only and do not automatically mutate weights, rules, thresholds, or models.

8.8 Judge Takeaway

KshetraAI closes the workflow loop by capturing field outcomes, while keeping future learning and recalibration controlled by humans.

9 FastAPI Backend Integration

9.1 Purpose

Expose the generated intelligence outputs through stable, schema-backed API endpoints for the frontend and demo workflow.

9.2 Implementation Summary

- Implemented a FastAPI backend endpoint.
- Added six stable API routes.
- Added Pydantic request and response schemas.
- Added service modules that format processed outputs into API responses.
- Added local frontend CORS support.
- Preserved a thin API boundary so route handlers do not duplicate intelligence logic.

9.3 How It Works

The API layer reads existing processed outputs and delegates response formatting to service modules. It does not recompute the full data pipeline, feature generation, prioritization, recommendation, anomaly, or explanation logic per request.

9.4 API Contract

Implemented endpoints:

```
GET /health
GET /daily-plan
GET /recommendations/{entity_id}
GET /alerts
GET /explanations/{entity_id}
POST /outcomes
```

Backend app metadata:

```
title: KshetraAI Backend
version: 0.1.0
```

Local frontend CORS origins:

```
http://127.0.0.1:5173
http://localhost:5173
```


9.5 Demo Evidence

Current fixed scenario API sample outputs are stored under:

demo/sample_outputs/

Files:

- daily_plan_response.json
- recommendation_response.json
- alerts_response.json
- explanation_response.json
- outcome_submission_response.json

Current daily-plan sample:

```
rep_id: REP_0164
territory_id: TER_0164
date: 2026-05-17
page_size: 3
total_count: 23
total_pages: 8
```

9.6 Swagger / OpenAPI

FastAPI provides interactive documentation by default when the backend server is running:

```
http://127.0.0.1:8000/docs
http://127.0.0.1:8000/openapi.json
```

This is framework-provided documentation, not a custom API documentation tool.

9.7 Verification

Relevant implementation areas:

- backend/main.py
- backend/api/routes/
- backend/api/services/
- backend/api/schemas/
- tests/test_build08_api_contract.py
- tests/test_build08_api_schemas.py
- tests/test_build08_fastapi_entrypoint.py
- tests/test_build08_planning_recommendation_routes.py
- tests/test_build08_alert_explainability_routes.py
- tests/test_build08_outcome_routes.py

Focused Build 08 API tests were verified during development.

9.8 Current Limits

- No authentication or role-based access control is implemented.
- No production database persistence is implemented.
- The API reads processed CSV outputs for the demo workflow.
- The backend is local-demo ready, not cloud-deployed.

9.9 Judge Takeaway

KshetraAI has a working FastAPI layer that exposes daily plan, recommendation, alerts, explanations, and outcome submission without mixing transport logic with intelligence logic.

10 Frontend Dashboard and Workflow Layer

10.1 Purpose

Turn backend intelligence outputs into a usable field-force workflow that a judge or representative can follow end to end.

10.2 Implementation Summary

- Built a React + TypeScript frontend using Vite.
- Implemented a dashboard workflow shell.
- Added pages for overview, daily plan, recommendation/explanation, alerts, and outcome submission.
- Added a typed API client for backend endpoints.
- Added hooks for API data loading.
- Added loading, error, empty, and success states.
- Added pagination for daily plan and alerts.
- Refined the UI with compact cards, status badges, and clearer action buttons.

10.3 How It Works

The frontend stores a selected rep, territory, date, and entity. It calls backend endpoints through one API client and renders the returned intelligence outputs without recalculating scores, recommendations, alerts, or explanations in the browser.

10.4 Frontend Workflow

Implemented workflow steps:

- Overview
- Daily Plan
- Recommendation
- Alerts
- Outcome

Default demo selection:

```
rep_id: REP_0164
territory_id: TER_0164
plan_date: 2026-05-17
selected_entity_id: RTL_01300
```

10.5 Demo Evidence

Frontend app files:

- `frontend/main.tsx`
- `frontend/App.tsx`
- `frontend/pages/Dashboard.tsx`
- `frontend/pages/VisitPlan.tsx`
- `frontend/pages/RecommendationView.tsx`
- `frontend/pages/AlertsView.tsx`
- `frontend/pages/OutcomeSubmission.tsx`
- `frontend/services/apiClient.ts`

Current UI behavior:

- daily plan page size: 3
- alerts page size: 3
- daily-plan cards show score, priority, and action button
- alert cards show score, severity, and confidence
- recommendation appears before explanation
- low / medium / high labels use shared color styling

10.6 Verification

Relevant implementation areas:

- `frontend/components/`
- `frontend/hooks/`
- `frontend/pages/`
- `frontend/services/apiClient.ts`
- `frontend/state/workflowStore.ts`
- `frontend/styles/global.css`
- `tests/test_build09_frontend_workflow.py`

Focused Build 09 tests and the frontend production build were verified during development:

```
python -m unittest tests.test_build09_frontend_workflow
npm run build
```

10.7 Current Limits

- The frontend does not implement authentication.
- The frontend does not include a map or route optimization view.
- The frontend does not compute intelligence logic; it displays backend outputs.
- Current frontend checks are Python static workflow tests plus TypeScript/Vite build, not a full browser automation suite.

10.8 Judge Takeaway

KshetraAI includes a working frontend workflow that makes the backend intelligence usable: plan, action, explanation, alert, and outcome.

11 Demo Integration, Testing, and Final Polish

11.1 Purpose

Package the implemented system into a deterministic, judge-facing demo path with supporting artifacts, verification scripts, and sanitized sample outputs.

11.2 Implementation Summary

- Created a deterministic demo scenario.
- Added a runbook, judging flow, presentation script, and checklist.
- Added a demo output generator.
- Added workflow verification and acceptance-check scripts.
- Generated sanitized API-level sample JSON outputs.
- Added Build 10 regression tests for demo readiness.

11.3 How It Works

The demo uses one fixed scenario, verifies that backend/frontend artifacts exist, checks that processed outputs support the scenario, and validates sample API payloads for the final judge-facing workflow.

11.4 Fixed Demo Scenario

Scenario file:

```
demo/scenarios/amritsar_crop_protection_scenario.md
```

Scenario identity:

```
scenario_id: AMRITSAR_CROP_PROTECTION_001
rep_id: REP_0164
territory_id: TER_0164
date: 2026-05-17
selected_entity: RTL_01300
```

Demo path:

Dashboard -> Daily Plan -> Recommendation -> Explanation -> Alerts -> Outcome

11.5 Demo Evidence

Sample API-level outputs:

- `demo/sample_outputs/daily_plan_response.json`
- `demo/sample_outputs/recommendation_response.json`
- `demo/sample_outputs/alerts_response.json`
- `demo/sample_outputs/explanation_response.json`
- `demo/sample_outputs/outcome_submission_response.json`

Current sample facts:

- top daily-plan entity: `RTL_01300`
- fixed scenario daily-plan rows: 23
- recommendation rows for top entity: 3
- explanation rows for top entity: 6
- alert rows for fixed territory: 19
- outcome response status: `success`

11.6 Verification

Demo verification scripts:

```
python demo\scripts\verify_demo_workflow.py
python demo\scripts\run_acceptance_checks.py
```

Current checks verify:

- required demo docs exist
- required frontend workflow files exist
- backend API route contract is present
- processed outputs exist
- fixed scenario rows exist
- top entity is stable
- sample output files exist
- sample payload shapes are valid
- configured forbidden raw-field terms are not present in sample outputs

Relevant tests:

- `tests/test_build10_demo_workflow.py`
- `tests/test_build10_acceptance_checks.py`

11.7 Current Limits

- The demo is local and deterministic, not cloud-deployed.
- Screenshots are not the primary committed evidence; sample API outputs and the live frontend/backend path are.
- The sanitization check is a configured demo-artifact check, not a complete enterprise privacy audit.
- Build 10 does not add new intelligence behavior; it integrates and verifies the existing system.

11.8 Judge Takeaway

KshetraAI is packaged with a repeatable demo scenario, verified outputs, and acceptance checks that connect the data, backend intelligence, API, frontend, and outcome workflow.

KshetraAI

Explainable. Deterministic. Human-governed.

Thank you.

Stage 1 Submission

Syngenta Hackathon

May 2026
